

Nama : Muhammad Arief Saputra
Kelompok : Bismuth

PROBLEM 1: The Last Astronaut

PROGRAM TheLastAstronaut

DECLARE:

N, K, posisi_saat_ini, nomor_tereliminasi : INTEGER
list_astronot : ARRAY OF INTEGER

START:

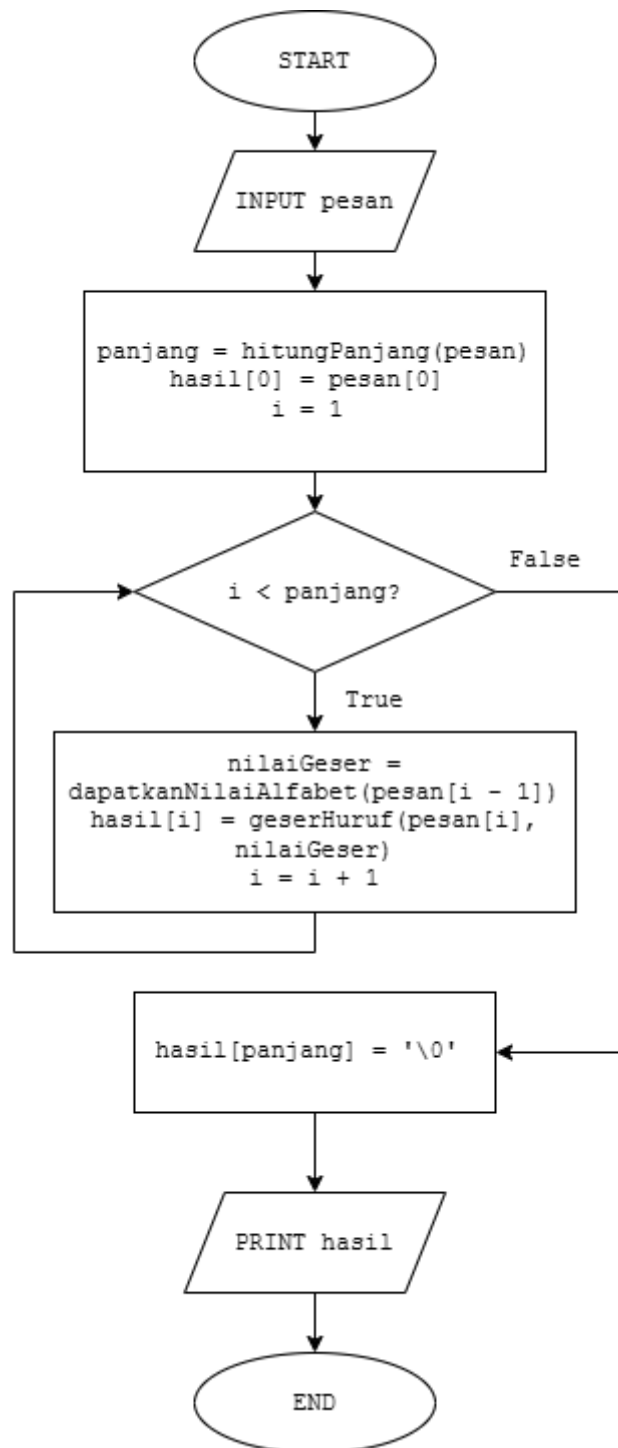
```
    INPUT N, K
    FOR i FROM 0 TO (N-1) DO
        SET list_astronot[i] = i + 1
    ENDFOR
    SET posisi_saat_ini = 0
    WHILE (jumlah elemen list_astronot > 1) DO
        SET posisi_saat_ini = (posisi_saat_ini + K - 1) MOD
jumlah elemen list_astronot
        SET nomor_tereliminasi =
list_astronot[posisi_saat_ini]
        PRINT "Astronot yang di-kick: ", nomor_tereliminasi
        REMOVE list_astronot[posisi_saat_ini]
        IF (nomor_tereliminasi MOD 2 == 0) THEN
            SET K = K + 2
        ELSE
            SET K = K - 1
        ENDIF
        IF (K < 2) THEN
            SET K = 2
        ENDIF
    ENDWHILE
    PRINT "The Last Astronaut: ", list_astronot[0]
END
```

Penjelasan logika penyelesaian masalah:

1. Kita mendeklarasikan variabel-variabel yang akan digunakan beserta tipe datanya: INTEGER untuk angka tunggal (N , K , $posisi_saat_ini$, $nomor_tereliminasi$, $jumlah_elemen$), dan ARRAY OF INTEGER untuk daftar astronot ($list_astronot$).
2. Saat program dimulai, pengguna diminta menginput nilai N dan K . N adalah jumlah astronot yang ikut permainan, sedangkan K adalah jarak hitungan awal untuk menentukan siapa yang tereliminasi pertama kali.
3. Program memvalidasi nilai masukan N . Jika $N < 1$, permainan tidak dapat dijalankan, program menampilkan pesan peringatan, dan meminta pengguna memasukkan ulang nilai N yang sesuai ($N \Rightarrow 1$). Jika $N = 1$, program langsung menampilkan The Last Astronaut adalah 1 tanpa meminta nilai K . Jika $N > 1$, program menginisialisasi array $list_astronot$ berisi angka 1 hingga N pada indeks 0 hingga $N-1$ (di mana astronot nomor $i+1$ disimpan pada indeks i).
4. Program memvalidasi nilai masukan K . Jika $K < 1$, proses eliminasi belum dapat dijalankan dan program melakukan perulangan (looping) untuk meminta pengguna memasukkan ulang nilai K yang sesuai ($K \Rightarrow 1$).
5. Kita tetapkan $jumlah_elemen\ awal = N$, dan $posisi_saat_ini = 0$ sebagai titik mulai (dalam bentuk indeks).
6. Selama jumlah elemen dalam array masih lebih dari 1, program terus melakukan proses eliminasi. Di setiap iterasi:
 - $posisi_saat_ini$ dihitung ulang dari posisi saat ini sebelumnya (bukan selalu dari 0 – hanya di iterasi pertama posisinya 0), ditambah $K - 1$, lalu di-MOD dengan $jumlah_elemen$.

- Alasan -1: jika posisi saat ini dianggap sebagai hitungan pertama, maka untuk mencapai hitungan ke-K kita hanya perlu melangkah sebanyak $K - 1$ langkah dari posisi itu (bukan K langkah).
 - Alasan MOD jumlah_elemen: supaya hitungan berputar melingkar begitu melewati elemen terakhir array. Contoh: elemen tersisa 4 (indeks 0-3), $K = 5$, maka $\text{posisi} + K - 1 = 5$, dan $5 \text{ MOD } 4 = 1$ – hitungan yang "lewat" dari indeks terakhir otomatis kembali berputar ke depan array, bukan keluar dari batas array.
7. nomor_tereliminasi diisi dengan nilai yang ada di `list_astronot[posisi_saat_ini]`. Setelah nilai itu didapat, program mencetak (PRINT) nomor astronot yang akan dieliminasi – ini langkah terpisah sebelum proses penghapusan.
 8. Astronot pada `posisi_saat_ini` dihapus dari array (REMOVE), lalu `jumlah_elemen` diperbarui (dikurangi 1).
 9. Nilai K diperbarui berdasarkan `nomor_tereliminasi`: jika genap, $K = K + 2$; jika ganjil, $K = K - 1$. Setelah itu dicek lagi: jika hasil K yang baru ternyata kurang dari 2, maka K dipaksa menjadi 2, agar permainan tidak macet/berulang di posisi yang sama.
 10. Proses di langkah 5-8 berulang terus selama elemen dalam array masih lebih dari 1. Ketika akhirnya hanya tersisa 1 elemen, program berhenti dan mencetak nomor yang tersisa di `list_astronot[0]` sebagai "The Last Astronaut".

PROBLEM 2: Alien-In-The-Middle



Penjelasan logika penyelesaian masalah:

Program diawali dengan menerima input pesan dari pengguna, kemudian menghitung total panjang karakter serta menyalin karakter pertama (pesan[0]) secara langsung ke

hasil[0] tanpa perubahan. Langkah inisialisasi ini juga menyiapkan indeks perulangan `i = 1` untuk memproses karakter-karakter berikutnya.

Selama kondisi perulangan `i < panjang` bernilai `True`, program mengambil nilai urutan alfabet dari karakter sebelumnya (`pesan[i - 1]`) sebagai besaran `nilaiGeser`. Karakter saat ini (`pesan[i]`) lalu digeser berdasarkan nilai tersebut dan disimpan ke `hasil[i]`, diikuti penambahan nilai `i` sebesar 1. Ketika kondisi perulangan bernilai `False`, program menambahkan penanda akhir string (`'\0'`), mencetak hasil enkripsi ke layar, dan program berakhir.